



HACKATHON PROJECT CONCEPT

ROCm Migration Copilot

An agentic assistant that ports CUDA code to HIP/ROCm: analyzing repositories, translating kernels, verifying numerical parity, and benchmarking workloads on AMD Instinct MI300X through the AMD Developer Cloud.

Team ROCm Rangers · Lead AI/ML Engineer

Agentic AI · ROCm 6.x · Open Source

Lablab x AMD AI Academy Challenge

September 1 — December 1, 2026

1. Executive Summary

The ROCm Migration Copilot is an open-source, agent-based developer tool that converts CUDA repositories into working, performance-verified HIP/ROCm ports. Given a GitHub repository, the agent analyzes the codebase, translates CUDA kernels and API calls to HIP, compiles and runs the result inside a ROCm 6.x Docker sandbox, verifies numerical parity against saved CUDA baseline outputs, and finally benchmarks the migrated workloads on an AMD Instinct MI300X provisioned through the AMD Developer Cloud. The output is a migration report with a confidence score, parity evidence, and measured speedups, all produced while a human approves every risky change through a lightweight review interface. In short, the project turns a multi-week manual porting grind into a supervised, repeatable agent workflow.

\$5,000 Total prize pool for the 3-month program	\$100 AMD Developer Cloud GPU credits per new member	3 Monthly build challenges, Sept 1 - Dec 1, 2026
--	--	--

The concept is deliberately engineered around the scoring system of the Lablab x AMD AI Academy Challenge. The Building category is the most heavily weighted stream of the program, and this project touches nearly every one of its line items: it is built on AMD technologies (+250 XP), submitted as a complete project (+200), released open source (+200), demonstrated with a working demo (+150), and structured to hit three explicit milestones (+300). Around the code, the program equally rewards learning and sharing, so the roadmap bakes in AI Academy courses, ROCm certification, blog posts, a demo video, and a case study from day one.

By December 1 the team expects to have a public GitHub repository with a working end-to-end agent, a recorded demo, at least three published content pieces, and a small gallery of real repositories successfully migrated. That combination is a portfolio strong enough for AMD showcase consideration, a credible asset for the closing final challenge, and a reusable tool the ROCm community can adopt the day the program ends.

2. Problem and Opportunity

CUDA has been the default programming model for GPU computing for nearly two decades, and the vast majority of research code, reference implementations, and performance-critical kernels still ship CUDA-first. For developers evaluating AMD's Instinct accelerators, that inertia is the single largest adoption barrier: before a single benchmark can run on a MI300X, existing code must be translated from CUDA to HIP, rebuilt against ROCm libraries, and validated. The port itself is rarely conceptually hard, because HIP is deliberately close to CUDA, but it is tedious, error-prone, and full of small mechanical decisions around API equivalents, library substitutions, and build-system changes that together consume days of skilled engineering time.

The tooling that exists today only partially helps. HIPIFY and similar translators perform syntax-level conversion of individual source files, but they do not understand a repository as a whole, they cannot close the loop by compiling, running, and testing their own output, and they provide no signal about whether the migrated code still computes the right answer or runs fast. Documentation is fragmented across the HIP porting guide, API equivalence tables, ROCm release notes, and scattered community threads. Verification is left entirely to the developer: saving baseline outputs, diffing results, and judging whether a numerical drift matters. As a result, migrations tend to stall not on the hard kernels but on the long tail of small fixes and unverified assumptions.

The opportunity is timing. ROCm 6.x is mature, vLLM and PyTorch run well on Instinct GPUs, and every new participant in this challenge receives \$100 of AMD Developer Cloud credits that remove the hardware barrier entirely. Open large language models are now capable enough to drive a function-calling agent through exactly this kind of iterative, tool-heavy engineering loop. AMD has a clear business interest in lowering migration friction, which makes this project unusually well aligned with the sponsor's roadmap: every successful migration the agent completes doubles as a demonstration, a benchmark data point, and shareable content for the Build-in-Public XP stream.

3. Solution Overview

The Copilot treats migration as a closed loop rather than a one-shot translation. A repository enters the pipeline, a migration plan comes out, the agent executes that plan in small reviewable steps, and every step is checked by a compiler, a test suite, and a benchmark before it is considered done. The pipeline has five stages, each exposed to the agent as a distinct tool through a function-calling interface, so the model decides when to call what while the tooling guarantees that nothing ships unverified.

Stage	What it does	Output
1. Repo Analyzer	Parses the repository: detects CUDA kernels, library calls, build system, and tests	Migration plan
2. Kernel Translator	Rewrites kernels and API calls to HIP using the mapping knowledge base	HIP source patches
3. Build-Fix Loop	Compiles in the ROCm sandbox, captures errors, and iterates until the port builds and runs	Working HIP build
4. Numerical Verifier	Runs parity tests against saved CUDA baseline outputs	Parity pass/fail with error metrics
5. Benchmark Reporter	Times the ported workloads on MI300X and assembles the run report	Confidence score and speedups

Table 1: The five-stage migration pipeline

Every completed run produces a migration confidence score: a composite of build success, numerical parity (maximum absolute error and allclose rates against the CUDA baselines), workload coverage, and benchmark stability. The score gives a developer an at-a-glance answer to the only question that matters, namely whether the port can be trusted, and it gives the agent a target to optimize across iterations. Risky transformations always route through the review UI, where a human approves or rejects diffs before they are applied, which keeps the agent squarely in copilot territory rather than autonomous rewriter territory.

Scope discipline matters in a three-month program, so the project carries explicit non-goals. The Copilot does not attempt to replace proprietary CUDA-only frameworks with functionally different stacks, and it does not target multi-node distributed systems in version one. It focuses on single-GPU, kernel- and library-heavy repositories where the CUDA-to-HIP mapping is well defined (cuBLAS to hipBLAS, cuDNN to MIOpen), and where a verified, benchmarked port is a realistic outcome within the program window.

4. Architecture and Technology Stack

The agent core is a Python function-calling loop in which one open-weight LLM alternates planner, coder, and reviewer roles under strict tool schemas, keeping the loop transparent and hackable. The knowledge base is a retrieval-augmented index built from the official HIP porting guide, API equivalence tables, ROCm documentation, and, crucially, every fix the agent has ever accepted, so the system improves with each migrated repository. Inference for the agent itself runs on open models served with vLLM on MI300X instances, which means the tool dogfoods the exact hardware and software stack it exists to promote.

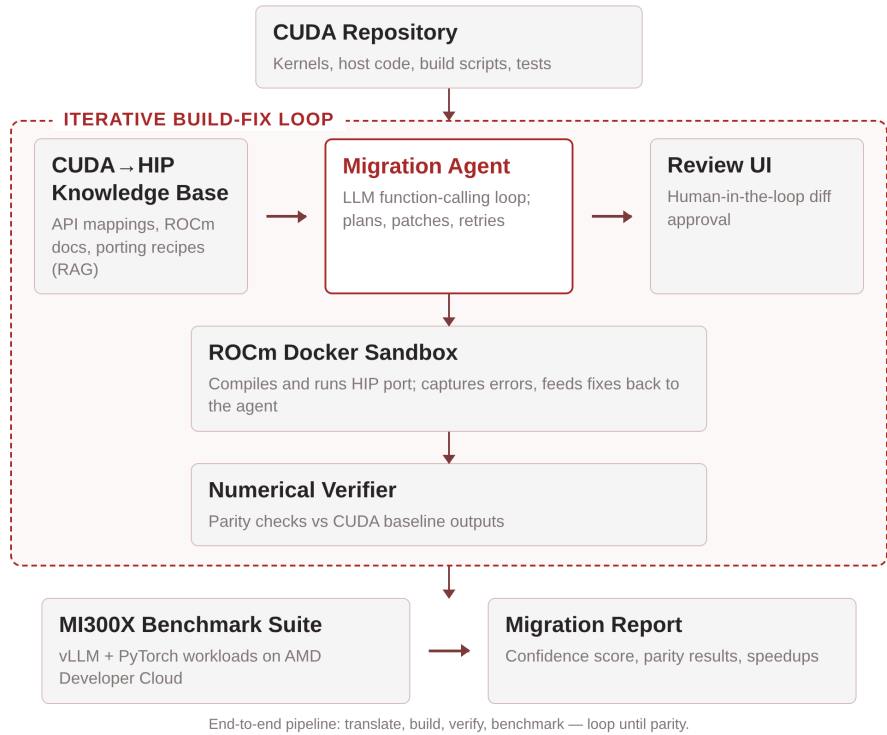


Figure 1: ROCm Migration Copilot architecture — the agent loops through translate, build, and verify until numerical parity is reached.

Layer	Technology	Why it is chosen
Agent orchestration	Python function-calling loop (planner / coder / reviewer)	Transparent, hackable, easy to demo live
Model inference	Open-weight LLMs served with vLLM on MI300X	Dogfoods the AMD stack the tool promotes
Knowledge base	Vector index of HIP porting guide, API tables, accepted fixes	Grounded translations that improve over time
Verification sandbox	Docker image with ROCm 6.x, HIP compilers, PyTorch	Reproducible compile-and-run checks per run
Benchmarking	Timing harness on AMD Developer Cloud MI300X	Comparable speedup numbers per repository
Demo interface	FastAPI backend with a React review dashboard	Human approval of every risky diff
Delivery	Public GitHub repository with CI	Open-source XP, community trust, forkability

Table 2: Technology stack by layer

Everything executes against the AMD Developer Cloud. Each agent run provisions an isolated Docker sandbox with ROCm 6.x preinstalled; compile errors, test logs, parity reports, and timings are archived as structured artifacts per run. Those artifacts are deliberately first-class outputs: they feed the confidence score, they become the evidence base for blog posts and the case study, and they give the demo video something real to show. A FastAPI backend plus a small React dashboard exposes runs, diffs, and scores for human review, and the whole system ships as an open-source repository with continuous integration so that anyone can point it at their own CUDA code.

5. Three-Month Execution Roadmap

The program runs September 1 to December 1, 2026. Month 1 is officially an onboarding and learning month with no project judging, and the build themes for Months 2 and 3 had not been announced at the time of writing. The roadmap therefore front-loads deterministic work (setup, learning, scaffolding) into September, keeps October and November milestone-driven, and reserves roughly one week of buffer in each build month to re-scope, so that whatever the monthly theme turns out to be, the Copilot's milestones can be submitted against it without derailing the core build.

Month	Program focus	Project milestones	XP hooks
September (Month 1)	Onboarding: connect accounts, claim credits, AI Academy courses, ROCm certification, first MI300X workload	Repository scaffolded; Repo Analyzer prototype; first MI300X job runs end to end	Onboarding 150; courses 250; certification 150; first project 55
October (Month 2, theme TBA)	Monthly build challenge; align Copilot scope to the announced theme	M1: minimum viable loop (analyzer, translator, build-fix) on a toy CUDA repo; demo submitted	Demo 150; milestone 100; AMD hackathon 300 if available
November (Month 3, theme TBA)	Final build month; final challenge unlocks in the closing weeks	M2: numerical verifier plus benchmark harness; M3: open-source polish, demo video, case study	Complete 200; open-source 200; milestones 200; video and case study 400

Table 3: Program alignment, September to December 2026

Three formal milestones anchor the build months. M1 at the end of October is a minimum viable loop in which the analyzer, translator, and build-fix stage work end to end on a small public CUDA repository. M2 in mid-November adds the numerical verifier and the MI300X benchmark harness, turning the tool from a translator into a migration system. M3 at the end of November is the open-source polish pass: documentation, a one-click demo, the recorded video, and the written case study. Each milestone lands

before the final-challenge window, leaving the closing weeks free for the Legend push rather than feature work.

6. XP Maximization and Build-in-Public Strategy

Because the challenge scores individual accounts, the XP plan treats the project as an engine that emits scoreable events on a calendar rather than as a single end-of-program submission. The table below maps every planned action to its XP value and target window. The cadence keeps the Build-in-Public stream honest: a short X or LinkedIn post every Sunday, one substantial blog or tutorial per month, and the video plus case study timed to the November milestones. Planned actions alone project roughly 2,475 XP, which clears Silver outright and presses into Gold (3,000+) once community answers, upvote bonuses, and referrals are counted.

Category	Action	XP	Window
Onboarding	Join Discord and complete the AMD profile	150	Week 1
Learning	AI Academy courses, incl. 5-course bonus	250	September
Learning	ROCm certification	150	September
Building	First project plus connected GitHub repo	55	September
Building	Build using AMD technologies	250	October
Building	Submit project demo	150	October
Building	Project milestones (3 × 100)	300	Oct - Nov
Building	Submit complete project	200	November
Building	Submit open-source project	200	November
Build in Public	Weekly X / LinkedIn posts (about 12)	120	Weekly
Build in Public	Demo video and case study	400	November
Knowledge Hub	Blog article and tutorial	250	Oct - Nov
Planned total	Before community, upvote, and referral bonuses	2,475	-

Table 4: Full XP map for one participant (founder)

Two rules keep the plan resilient. First, every XP-bearing action must also serve the project: content is generated from real run artifacts, never manufactured, which is exactly what the Build-in-Public verification

rules reward. Second, verification windows: the program dashboard is checked weekly, and any category lagging behind its calendar target triggers a scope adjustment at the next planning session rather than a scramble in the final week. Legend rank additionally requires the final challenge that unlocks near the end of the program, and the roadmap deliberately leaves November's closing weeks free for exactly that push.

7. Team Formation

The challenge scores XP on individual accounts only, so the team model is a working group rather than a scoring entity: everyone who joins earns XP on their own account from the same shared repository, which multiplies each member's Building XP without diluting anyone's score. The founder acts as lead AI/ML engineer, owning the agent loop and the overall architecture, and is recruiting LLM/ML engineers through the lablab Teams page and Discord to take ownership of defined modules: the CUDA-to-HIP knowledge base, the parity test harness, and the benchmark tooling.

Expectations are deliberately low-friction to maximize the join rate: five to ten hours per week, fully asynchronous, GitHub-first collaboration, and a stated bias toward motivation over prior ROCm experience, since the learning path (AI Academy courses and ROCm certification) is itself part of the program. Every new joiner receives a module, a milestone, and a content slot, which means each member can generate an individual XP stream from day one while the shared repository compounds in the background.

Appendix: Ready-to-Paste Team Description (992 characters)

The text below is the exact team-forming description for the lablab Teams page, held under the 1,000-character limit so it can be pasted without edits. It is written to be copied verbatim; feel free to add emoji or links when posting it on the platform.

Team ROCm Rangers is forming for the Lablab x AMD AI Academy Challenge! We're building ROCm Migration Copilot: an open-source AI agent that ports CUDA code to HIP/ROCm. Point it at a repo: it translates kernels, compiles in a ROCm sandbox, verifies results vs the CUDA baseline, benchmarks on AMD Instinct MI300X. No vendor lock-in, no weekend-long porting pain. Stack: Python, LLM function-calling loops, ROCm 6.x, PyTorch, vLLM, Docker, React, on AMD Developer Cloud credits. I'm the lead AI/ML engineer driving architecture and roadmap, seeking LLM/ML engineers to own modules: agent loop, CUDA-to-HIP knowledge base, parity harness, benchmark tooling. Motivation beats experience; if GPUs, agents, or AMD's ecosystem excite you, you'll learn fast. We build in public: weekly posts, demo video, blogs, a case study. XP is individual, but the repo, the wins, and the story are shared. Join early, shape the roadmap - let's ship what AMD will want to showcase. Find me on the lablab Discord!